# MoneyPoly White-Box Testing Report

Repository: `https://github.com/Athmeeya2006/dass-assignment-2`

## 1. Introduction

This report documents the white-box testing of the MoneyPoly board game implementation. The internal logic of the program was analysed using control-flow reasoning, branch testing, state-based testing, and edge-case testing. The main goal was to detect logical defects in movement, money handling, ownership, card processing, jail flow, bankruptcy, and winner selection.

## 2. Program Overview

MoneyPoly is a command-line board game that manages players, board movement, taxes, property purchasing, rent payment, chance and community chest cards, jail behavior, bankruptcy, and final winner selection.

## 3. Control Flow Graph

**Manual submission item:** the assignment requires a hand-drawn control flow graph image. It should be placed in `whitebox/diagrams/` and attached to the final report package. This PDF records the intended CFG scope: program start, player input, game setup, the main loop, turn execution, jail handling, tile resolution, card handling, bankruptcy checks, and winner selection.

## 4. Code Quality Analysis

`pylint` was used iteratively to improve the quality of the MoneyPoly codebase. The issues addressed included unused imports, broad exception handling, style inconsistencies, boolean-comparison cleanup, long lines, missing module/class/function docstrings, and final design-level cleanup.

**Verified result:** `PYLINTHOME=/tmp/pylint .venv/bin/python -m pylint whitebox/code/moneypoly/main.py whitebox/code/moneypoly/moneypoly/*.py` produced **10.00/10**.

- Iteration 1: Remove unused imports and replace broad exception handling
- Iteration 2: Simplify boolean checks and tidy control-flow style issues
- Iteration 3: Add missing module, function, and class docstrings
- Iteration 4: Wrap long card definitions and normalize formatting
- Iteration 5: Refactor card handling and resolve remaining pylint design warnings
- Iteration 6: Add automated Git commit helper and track the whitebox report

## 5. White-Box Test Design

The white-box tests were designed from the structure of the implementation rather than only from input/output behavior. The suite targets all major branches, key variable states, and important edge cases such as empty decks, exact-balance purchases, invalid trades, passing Go, mortgage failures, railroad handling, and bankruptcy progression.

**Verified result:** `python3 -m pytest whitebox/tests -q` produced **35 passed**.

| ID | Test Case | Scenario | Why It Was Needed | Error or Issue Found |
|---|---|---|---|---|
| TC1 | Board tile classification | Verifies routing through go, property, community chest, and blank tiles. | Confirms _move_and_resolve depends on correct tile classification. | No new logic bug; used as a structural branch check. |
| TC4 | Empty deck safety | Exercises draw, peek, cards_remaining, and repr on an empty CardDeck. | Covers the empty-collection edge case. | cards_remaining() and __repr__() crashed on empty decks. |
| TC5 | Setup validation | Checks single-player and duplicate-name | The game requires at least two unique | Game setup accepted invalid player lists. |

| | | | | |
|---|---|---|---|---|
| | | game creation. | players. | |
| TC6 | Dice range | Checks that both dice use the full 1..6 range. | All movement paths depend on correct dice boundaries. | Dice used 1..5 instead of 1..6. |
| TC7 | Passing Go salary | Moves a player across the board boundary. | Boundary movement affects core money logic. | Salary was only awarded when landing exactly on Go. |
| TC8 | Full group ownership | Checks that rent only doubles when the full group is owned. | Rent multiplier depends on complete ownership, not partial ownership. | PropertyGroup.all_owned_by() used partial ownership logic. |

| | | | | |
|---|---|---|---|---|
| TC9 | Bank accounting | Checks loan payout and negative bank collection handling. | Money-transfer consistency is critical to the game. | Loans did not reduce bank reserves, and negative collects changed bank funds. |
| TC10 | Purchase boundaries | Checks exact-balance purchase and rejection of owned or mortgaged properties. | Covers a boundary value and misuse of direct purchase methods. | Exact-balance purchase failed, and invalid purchases were allowed. |
| TC11 | Rent transfer | Checks tenant debit and owner credit in one transaction. | Rent must update both participants. | Rent was deducted from the tenant but not credited to the owner. |
| TC12 | Mortgage consistency | Checks mortgage/unmortgage behavior when funds are | State must not change before affordability checks pass. | Mortgage and unmortgage could leave inconsistent property state. |

| | | | | |
|---|---|---|---|---|
| | | insufficient. | | |
| TC13 | Trade validation | Checks seller credit, negative cash rejection, and self-trade rejection. | Trades affect money, ownership, and inventories. | Seller was not credited; invalid trade inputs were accepted. |
| TC14 | Jail fine deduction | Checks voluntary release from jail. | Jail release is a money-state branch. | The bank was credited but the player was not charged. |
| TC16 | Special tile routing | Covers go_to_jail, income tax, luxury tax, and free parking. | These are major decision branches in _move_and_resolve. | Used to verify money and jail state transitions. |
| TC17 | Card action branches | Covers collect, pay, jail, jail_free, move_to, birthday, and collect_ | Card logic contains several internal branches and state transitions. | Confirmed multiple card-action paths and poor-player skip behavior. |

| | | | | |
|---|---|---|---|---|
| | | from_all. | | |
| TC18 | Railroad routing | Checks that railroad tiles map to real property objects. | Without railroad property objects, the railroad branch does nothing. | Railroad tile positions had no corresponding Property objects. |
| TC19 | Bankruptcy clean up | Checks elimination, property reset, and player removal. | Bankruptcy is a major state transition. | Used to confirm cleanup logic and downstream turn handling. |
| TC20 | Net worth and winner selection | Checks property value inclusion and richest-player selection. | Standings and winner logic depend on accurate net worth. | net_worth() ignored properties, and winner selection used min instead of max. |
| TC23 | Turn order after bankruptcy | Checks next-player progression and doubles logic after eliminat | Removing the current player can corrupt turn flow. | Bankruptcy could skip the next player or incorrectly preserve an extra turn. |

ion.

## 6. Error Fixes Applied

The defects revealed by testing were fixed one by one and saved using the required `Error #:` commit format. The verified fixes covered setup validation, dice range, passing-Go salary, full-group ownership, bank accounting, empty card decks, exact-balance purchases, purchase validation, rent transfer, mortgage consistency, trade validation, jail fine deduction, auction validation, railroad properties, interactive menu reachability, net-worth calculation, winner selection, and bankruptcy turn-flow behavior.

**Commit workflow completed:** test1; Error 1 through Error 21; Iteration 1 through Iteration 6

## 7. Summary of Verified Logic Corrections

- Game setup now rejects fewer than two players and duplicate names.
- Dice rolls use the full six-sided range.
- Crossing Go correctly awards salary.
- Color-group rent bonuses only apply to full ownership.
- Bank loans and collections now preserve money consistency.
- Exact-balance purchases are allowed and invalid purchases are rejected.
- Rent, trade, mortgage, unmortgage, and jail fine logic all update balances correctly.
- Railroad tiles now resolve to actual property objects.
- Winner selection uses the highest net worth, and net worth includes property values.
- Bankruptcy no longer breaks turn order or doubles behavior.

## 8. Conclusion

The white-box testing exercise revealed several important logical issues in the original MoneyPoly implementation, especially in financial consistency and turn progression. After the targeted fixes, the code passed all designed white-box tests and achieved a full `pylint` score of `10.00/10`.